

Targeted Mining of Top- k High Utility Itemsets

Shan Huang^a, Wensheng Gan^{a,b,*}, Jinbao Miao^a, Xuming Han^c and Philippe Fournier-Viger^d

^aCollege of Cyber Security, Jinan University, Guangzhou 510632, China

^bPazhou Lab, Guangzhou 510330, China

^cCollege of Information Science and Technology, Jinan University, Guangzhou 510632, China

^dCollege of Computer Science and Software Engineering, Shenzhen University, Shenzhen 518060, China

ARTICLE INFO

Keywords:
data science
data mining
utility mining
target itemset
targeted mining
top- k

ABSTRACT

Finding high-importance patterns in data is an emerging data mining task known as High-utility itemset mining (HUIM). Given a minimum utility threshold, a HUIM algorithm extracts all the high-utility itemsets (HUIs) whose utility values are not less than the threshold. This can reveal a wealth of useful information, but the precise needs of users are not well taken into account. In particular, users often want to focus on patterns that have some specific items rather than find all patterns. To overcome that difficulty, targeted mining has emerged, focusing on user preferences, but only preliminary work has been conducted. For example, the targeted high-utility itemset querying algorithm (TargetUM) was proposed, which uses a lexicographic tree to query itemsets containing a target pattern. However, selecting the minimum utility threshold is difficult when the user is not familiar with the processed database. As a solution, this paper formulates the task of targeted mining of the top- k high-utility itemsets and proposes an efficient algorithm called TMKU based on the TargetUM algorithm to discover the top- k target high-utility itemsets (top- k THUIs). At the same time, several pruning strategies are used to reduce the memory consumption and execution time. Extensive experiments show that the proposed TMKU algorithm has good performance on real and synthetic datasets.

1. Introduction

As digital systems are widely used, the era of big data has arrived, with massive amounts of data being generated. Therefore, figuring out how to mine useful information from complex data has become a major issue. Data mining software assists clients in finding correlations in millions or billions of records, allowing decision-makers to make more informed decisions faster [10, 15, 18]. This technology is influencing many areas of daily life (e.g., market analysis [21], biomedicine [31], and network security management [4]). A typical application is market basket analysis [25]. Market basket analysis primarily analyzes the consumption records of customers to uncover important correlations between products and services, allowing them to rearrange products on shelves or design a combination of promotional packages that users are interested in. For example, if a supermarket's sales records show that users often buy bread and milk at the same time, bundling them on the same shelf can increase sales and revenue.

Previous studies have proposed numerous frequent itemset mining algorithms (FIM) [1, 25], among which the most famous are Apriori [3] and FP-growth [20]. Traditional frequent pattern mining algorithms use only one metric, support, to find patterns that users are interested in. As a result of applying the frequent itemset model, itemsets are often found to have a high selling frequency but yield a low profit. But some luxury goods that are not widely sold are highly profitable and deserve attention. To address this issue, utility-oriented pattern mining (UPM) [19, 22, 34, 41] algorithms have been proposed. Internal and external utilities are both included in the concept of utility. The internal utility usually refers to how many times a good appears, while the profit per unit generated by each good is known as the external utility. UPM has been extensively studied in recent decades [16, 17], since it is beneficial to consider the profitability and relative importance of each item. Some representative UPM algorithms are HUI-Miner [28], which is based on the utility-list structure; UP-Growth [39] based on the UP-tree structure; and EFIM [45] based on data projection.

Although high-utility itemset mining algorithms are able to collect some useful information in specific practical applications, they are not suitable for target-oriented mining tasks. The targeted high-utility itemset querying (TargetUM)

*Corresponding author

shuang9901@gmail.com (S. Huang); wsgan001@gmail.com (W. Gan); osjbmiao@gmail.com (J. Miao); hanxuming@jnu.edu.cn (X. Han); philfv@qq.com (P. Fournier-Viger)

algorithm [30] was introduced to resolve this issue by giving consideration to user needs. Take consumer purchases as an example: people pay more attention to the items on their shopping list and less to other items. Therefore, retailers can offer different product recommendations according to the various needs of users as well as discounts on goods in stock. Hence, target-oriented high-utility itemset mining (THUIM) can filter out many irrelevant patterns in the data to precisely focus on those containing items of interest. However, it is complex and laborious to select the minimum utility threshold in these utility-driven data mining algorithms. In particular, if a user is not familiar with a database, he will not have a good understanding of the utilities of itemsets and the content of the transactions. When the utility threshold is set too high, only a small percentage of target high-utility patterns are captured, while a lot of irrelevant information will be generated for a low utility threshold, which has no value to the user. As a result, depending on the number of items and utility distribution in each database, different utility thresholds must be set. In general, decision makers will always repeatedly test and modify the utility threshold until they find a threshold value that will produce the appropriate number of target high-utility itemsets, which can take a long time and is inconvenient.

Consequently, the problem of targeted mining of top- k high utility itemsets is formulated in this paper, for which the user can directly set the number of itemsets to be mined. An algorithm called TMKU (Targeted Mining of top- k high Utility itemset) is designed to perform this task efficiently. TMKU has proved effective and efficient in experiments. The main contributions of this study are outlined below:

- This paper considers two important aspects: utility mining and targeted pattern discovery. As far as we know, the TMKU algorithm is the first algorithm to incorporate the concept of top- k mining into target-based utility mining.
- A trie structure is used to query the target itemsets and a new data structure, namely TopKMap is used to efficiently store and retrieve the top- k target itemsets, which is beneficial to reduce the query time and search space.
- The TMKU algorithm utilizes several upper bounds. Three pruning strategies are used to reduce the search space. In addition, two target utility raising strategies are adopted to quickly raise the threshold and ensure that all top- k THUIs are discovered.
- Extensive experiments were conducted on both real and synthetic datasets. The results demonstrate that TKUM is efficient and effective.

The rest of this paper is composed of six sections. A brief review of related work is provided in Section 2. Section 3 describes the top- k THUIM problem and related definitions. Section 4 introduces the proposed TMKU algorithm, as well as the main strategies and techniques applied. The experimental results are presented and evaluated in Section 5. Ultimately, Section 6 concludes this paper and provides an outlook on promising research directions.

2. Related Work

The section introduces the research related to the paper, which can be divided into three parts: (1) high-utility pattern mining; (2) top- k utility itemset mining; and (3) targeted pattern mining.

2.1. High-utility pattern mining

Frequent itemset mining (FIM) [1, 3, 20] has been extensively studied for decades. However, relying only on frequency cannot bring enough benefits to users. Factors such as quantity and profit should also be considered. For this reason, Chen *et al.* [5] put forward a new task called high-utility itemset mining (HUIM). Since then, utility mining research has developed rapidly [17, 27, 35, 41]. For the convenience of discussion, high-utility itemset mining algorithms are grouped into the following three categories:

Apriori-based algorithms: Since Agrawal *et al.* [2] proposed the Apriori property in 1994, lots of algorithms based on Apriori have been published. For example, Liu *et al.* [29] introduced the prominent Two-Phase algorithm to handle the difficulty that the utility, unlike frequency, is neither monotone nor anti-monotone. That algorithm uses an overestimation of the utility called *TWU* (Transaction Weighted Utilization) to find candidate itemsets in a first phase. Thereafter, in a second phase, the database is searched again to determine the exact utility value of each candidate itemset. The IIDS algorithm [26] is an improved version of Two-Phase that discards isolated items to shrink the search

space. However, the common disadvantage of Apriori-like algorithms is that plenty of candidate patterns are generated, resulting in considerable computational costs and memory consumption.

Tree-based algorithms: Tseng *et al.* [39] designed the UP-tree structure, a utility-pattern tree, and introduced the UP-Growth algorithm inspired by FP-Growth. Subsequently, other versions of tree-based algorithms [34, 37] have been presented. In general, utilizing the UP-tree can prevent many meaningless database scans. When working with large-scale databases, however, this structure grows increasingly complex and occupies a massive amount of memory.

Other structure-based algorithms: HUI-Miner [28] utilizes a novel data structure known as a utility-list, which avoids the difficulty of generating numerous candidates. Moreover, the FHM algorithm [12] reduces the cost of join operations by using a tighter upper bound, which results in outperforming HUI-Miner. However, the join operation on lists of these algorithms takes time and memory. Thus, Zida *et al.* [45] proposed the EFIM algorithm with high-utility database projection (HDP) and high-utility transaction merging (HTM) techniques to lower the expensive cost of database passes. The utility-list-based CoUPM algorithm for correlated utility-based pattern mining [14]. In summary, these algorithms integrate various strategies to discover HUIs as efficiently as possible.

2.2. Top- k utility itemset mining

Although the above algorithms are effective in finding the desired set of itemsets, the efficiency of mining is strongly related to the selection of the minimum utility threshold. However, it is not easy to identify an appropriate threshold. Many top- k pattern mining algorithms were thus designed to directly discover the set of top- k HUIs, rather than asking users to specify a utility threshold. Top- k HUIM algorithms mainly consist of two types: the first is the two-phase algorithms, and the other is the one-phase algorithms.

Two-phase algorithms: The task of discovering the top- k HUIs was proposed by Wu *et al.* [40] with the TKU algorithm, which outperformed HUIM algorithms in terms of speed. The TKU algorithm is a two-phase algorithm. In the first phase, a UP-Tree is built, and promising top- k HUIs are generated. Then, in the second phase, the desired top- k HUIs are selected among them. TKU applies several strategies to filter unpromising candidates during the search [38] and achieve higher efficiency. Subsequently, REPT [32] was introduced with optimizations to record and pre-calculate the utility of items to prune the search space effectively and raise the minimum utility threshold. REPT uses a tree structure and pre-evaluation matrixes as tools to store utility information. However, these two-phase algorithms still generate large sets of candidates, which causes unreasonably long runtimes and high memory usage.

One-phase algorithms: For top- k HUIM, the one-phase TKO algorithm [38] was developed to solve the shortcomings of two-phase algorithms. TKO takes advantage of the utility-list structure of HUI-Miner, and outperforms the TKU and REPT algorithms according to experiments [38]. Similarly, another one-phase algorithm called KHMC [9] also discovers the top- k HUIs by using the utility-list structure. In KHMC, an estimated utility co-occurrence pruning (EUCP) technique is applied, which is based on precalculating the TWU of 2-itemsets. Moreover, the algorithm also adds another pruning strategy named early abandoning to avoid completely constructing the lists of unpromising itemsets. Three threshold-raising strategies are able to significantly shrink the search space and enhance the algorithm's efficiency. The THUI algorithm [23] has better performance thanks to introducing the concept of Leaf Itemset Utility (LIU), a triangular matrix, which can be implemented with only a small amount of memory to store utility information. Besides, the LIU-E and LIU-LB threshold raising strategies also accelerate the mining speed of the algorithm. THUI greatly outperforms TKO and KHMC, especially for dense or large datasets.

In addition, there are various other top- k pattern mining problems and variations, such as mining top- k sequential patterns [43], mining top- k HUIs in data streams [7], discover top- k high-utility sequential patterns [43], and mining top- k HUIs with negative utility values [36].

2.3. Targeted pattern mining

Those algorithms listed above are designed to find all itemsets that meet a single predetermined criterion. Target-oriented query algorithms give an alternative solution to this problem by filtering out unnecessary information. Rather than searching for numerous but mostly insignificant items, the user can enter any target and then discover patterns containing the desired items. Several target-oriented query algorithms based on frequency have been developed in earlier studies. These interactive methods are capable of returning results containing a target. Kubat *et al.* [24] were among the first to address the issue of processing target queries in a transactional database. They implemented target query processing algorithms for association mining by creating itemset trees that can be progressively updated. Fournier-Viger *et al.* [11] developed the Memory Efficient Itemset Tree (MEIT) to further reduce memory requirements. The tree is optimized to perform incremental modifications when new transactions are inserted, and it employs

a node-compression method. For multi-objective mining of big data, the guided FP-growth (GFP-growth) algorithm based on FP-Growth was proposed by Shabtay *et al.* [33]. In particular, many experiments have illustrated the excellent performance of the algorithm on imbalanced data. Target-oriented mining has also been studied and applied to discover sequential patterns. The targeted mining algorithm for sequential patterns proposed by Chueh *et al.* [8] speeds up the search for the target itemsets by using the reversion of the original sequence and comparing the reversed sequence with the related itemsets. Furthermore, clustering analysis is applied to automatically set time partition values for the task of time-interval sequential pattern mining. A novel target-oriented sequential pattern mining approach was presented by Chand *et al.* [6], which uses RFM (recency, frequency, and monetary) constraints. As a result, fewer database projections are done, and the space complexity is reduced. To remove some useless or irrelevant patterns in high utility sequential pattern mining [44, 13], the TUSQ algorithm [42] first introduced the concept of utility into target sequence queries. The algorithm does not focus on frequency like previous algorithms, but rather on utility. Recently, the TargetUM algorithm [30] has been proposed to fill the gap and perform target-oriented mining in HUIM.

In general, the TargetUM algorithm provides an integrated approach for high-utility mining with a target query, which serves as the foundation for this research. However, there are no studies combining top- k high-utility methods with target pattern queries. This paper introduces the problem of targeted utility mining with the concept of top- k patterns to prevent the generation of large sets of HUIs and to accurately and quickly process target queries.

3. Preliminaries and Problem Statement

This section introduces the redesigned data structure and briefly describes key definitions concerning the problem of targeted top- k high utility itemset mining. $\mathcal{D}$ denotes a transaction database represented as multiple transactions $\{t_1, t_2, t_3, \dots, t_n\}$. Each transaction contains one or more items from a set denoted as $\{i_1, i_2, i_3, \dots, i_m\}$. The transaction identifier of a transaction t_{id} is id . In the case where an itemset includes l items, it is called an l -itemset. Each item in a transaction can be annotated with a positive integer value indicating a purchase quantity (also called internal utility value or count). For example, $t_1 = \{(b, 4), (d, 3), (e, 1)\}$ represents four units of an item b , three units of an item d , and one unit of an item e . The itemset $\{b, d, e\}$ is called a 3-itemset.

Table 1: Example database

T_{id}	Transaction	Count
t_1	$\{b, d, e\}$	$\{4, 3, 1\}$
t_2	$\{a, b, e\}$	$\{1, 3, 2\}$
t_3	$\{c, d\}$	$\{3, 1\}$
t_4	$\{a, b, c, f, g\}$	$\{3, 1, 4, 2, 1\}$
t_5	$\{a, b\}$	$\{1, 2\}$
t_6	$\{b, c, d, f\}$	$\{1, 3, 4, 2\}$
t_7	$\{a, e, g\}$	$\{4, 1, 2\}$

Table 2: High-utility itemsets w.r.t $minUtil = \$25$

Itemset	Utility	Itemset	Utility
$\{f, c, d, b\}$	\$27	$\{c, d, b\}$	\$25
$\{e, d, b\}$	\$26	$\{d\}$	\$32
$\{e, b\}$	\$27	$\{d, b\}$	\$43
$\{c, d\}$	\$32	$\{b\}$	\$33

Definition 1. (The utility of the itemset) There are two types of utility values in $\mathcal{D}$. On the one hand, in each transaction t_{id} , the internal utility of an item $i \in t_{id}$ is denoted as $iu(i, t_{id})$. On the other hand, the external utility of an item i is a positive number denoted as $eu(i)$ that is utilized to indicate its relative weight or importance. Thus, in the transaction t_{id} , $u(i, t_{id})$ represents the utility of item i , which is the product of $eu(i)$ and $iu(i, t_{id})$, that is $u(i, t_{id}) = eu(i) \times iu(i, t_{id})$. For an itemset X and a transaction t_{id} containing X ($X \subseteq t_{id}$), the utility of X in t_{id} is calculated as $u(X, t_{id}) = \sum_{i \in X \wedge X \subseteq t_{id}} u(i, t_{id})$. The notation $u(X)$ denotes the utility of X in $\mathcal{D}$ and it is calculated as the sum of the utility of X in all transactions containing X , that is $u(X) = \sum_{X \subseteq t_{id} \in \mathcal{D}} u(X, t_{id})$.

For example, the external utility of all items from the example database are $\{(a: 1), (b: 3), (c: 2), (d: 4), (e: 2), (f: 1), (g: 2)\}$. In Table 1, $iu(b, t_2) = 3$, $eu(b) = 3$, then $u(b, t_2) = iu(b, t_2) \times eu(b) = 3 \times 3 = \9 . $u(\{ae\}, t_2) = u(a, t_2) + u(e, t_2) = 1 \times \$1 + 2 \times \$2 = \5 , $u(\{abc\}, t_4) = u(a, t_4) + u(b, t_4) + u(c, t_4) = 3 \times \$1 + 1 \times \$3 + 4 \times \$2 = \$14$. Since d has appeared in t_1, t_3 , and t_6 , $u(d) = u(d, t_1) + u(d, t_3) + u(d, t_6) = 3 \times \$4 + 1 \times \$4 + 4 \times \$4 = \$32$, and $u(\{ab\}) = u(\{ab\}, t_2) + u(\{ab\}, t_4) + u(\{ab\}, t_5) = u(a, t_2) + u(b, t_2) + u(a, t_4) + u(b, t_4) + u(a, t_5) + u(b, t_5) = 1 \times \$1 + 3 \times \$3 + 3 \times \$1 + 1 \times \$3 + 1 \times \$1 + 2 \times \$3 = \23 .

Definition 2. (The transaction-weighted utility of itemset) The notation $tu(t_{id})$ refers to the utility of the transaction t_{id} and is defined by the sum of the utilities of all the items in t_{id} . In other words, $tu(t_{id}) = \sum_{i_{\ell} \in t_{id}} u(i_{\ell}, t_{id})$. The transaction-weighted utility of an itemset X in $\mathcal{D}$ is denoted as $TWU(X)$ and is the sum of the utilities of X in all the transactions containing X in $\mathcal{D}$, that is $TWU(X) = \sum_{X \subseteq t_{id} \wedge t_{id} \in \mathcal{D}} u(X, t_{id}) = \sum_{X \subseteq t_{id} \wedge t_{id} \in \mathcal{D}} \sum_{i_{\ell} \in X} u(i_{\ell}, t_{id})$.

For example, $tu(t_1) = \$26$, and the transaction having the maximum utility is t_6 with $tu(t_6)$. Moreover, the calculated transaction utility of each transaction is $\{(t_1: \$26), (t_2: \$14), (t_3: \$10), (t_4: \$18), (t_5: \$7), (t_6: \$27), (t_7: \$10)\}$.

Property 1. The TWU complies with the property of downward closure. This means that if an itemset's $TWU(X)$ is less than or equal to the utility threshold ξ , any other itemsets containing X are low-utility itemsets.

The TWU values of all the items are determined by initially scanning the database $\mathcal{D}$. Property 1 states that when $TWU(X) < \xi$ for an item X , X will be discarded to reduce the number of itemsets to be evaluated. To achieve the goal of fast mining, the paper uses a TWU ascending sort and processes each transaction in the transaction database according to that order as well. The transaction-weighted utility of all 1-itemsets is sorted as: $g (= \$28) < f (= \$45) < a (= \$49) < e (= \$50) < c (= \$55) < d (= \$63) < b (= \$92)$.

Definition 3. (Remaining utility) If there is an itemset X , and $X \subseteq t_{id}$, the remaining items after X in t_{id} is denoted as t_{id}/X . The remaining utility [28] of itemset X in the transaction t_{id} is the sum of the utilities of t_{id}/X denoted as $ru(X, t_{id})$, where $ru(X, t_{id}) = \sum_{i_{\ell} \in t_{id}/X} u(i_{\ell}, t_{id})$. Similarly, the remaining utility of X in $\mathcal{D}$ is denoted as $ru(X)$, where $ru(X) = \sum_{X \subseteq t_{id} \in \mathcal{D}} ru(X, t_{id})$.

In Table 1, the remaining utility of $\{bd\}$ in t_1 is defined as $ru(\{bd\}, t_1) = u(e, t_1) = \2 .

Definition 4. (Utility-list) In HUI-Miner [28], the utility-list was introduced for the utility mining task. There are three essential elements (t_{id} , $iutil$, and $rutil$) in the utility-list. The t_{id} is the identifier of a transaction that contains X . Element $iutil$ represents the utility of X_i in t_{id} . Similarly, $rutil$ is the remaining utility of X_i in t_{id} .

For example, the utility-list of $\{c\}$ is $\{t_3, \$6, \$4\}$, $\{t_4, \$8, \$4\}$ and $\{t_6, \$6, \$18\}$. The utility-list of $\{f\}$ is $\{t_4, \$2, \$2\}$ and $\{t_6, \$2, \$0\}$. Therefore, the utility-list of $\{c, f\}$ is $\{t_4, \$10, \$2\}$ and $\{t_6, \$8, \$0\}$.

Definition 5. (Target high-utility itemset) Let $targetUtil$ be the target utility threshold specified by a user and denoted as ξ . X is known as a HUI for the user if its utility value is not smaller than ξ . A target itemset set by the user is called T' . This itemset is called target since it is what the user wants to focus on. If $u(X) > \xi$, $T' \subseteq X$, then X is further recognized as one of the target high-utility itemsets (THUIs) [30].

For example, a user wants to look for HUIs related to the target itemset $T' = \{b, d\}$. From Table 2, we can search the HUIs with T' . As a result, the set of THUIs is $\{\{f, c, d, b\}, \{e, d, b\}, \{c, d, b\}, \{d, b\}\}$.

Definition 6. (Top- k target high-utility itemset) Inspired by other top- k pattern mining tasks, we define the top- k target high-utility itemsets (top- k THUIs) as the k targeted itemsets that have the highest utility values. The result itemsets will be less than k items if the itemsets containing the target pattern in the database are relatively rare and the k set by the user is relatively large. When there are more than k patterns with the same utility, the results will also show only the k itemsets in the order of mining.

For example, if a user only wants to find the top-3 itemsets that satisfy the previous condition, then we set k to 3. Obviously, the top-3 target HUIs are $\{f, c, d, b\} = \$27$, $\{e, d, b\} = \$26$, and $\{d, b\} = \$43$.

Problem statement: Most utility mining algorithms are designed to identify itemsets with a utility equal to or greater than a fixed utility threshold. However, in reality, the user does not want to find all HUIs. For instance, a user may only want the top- k HUIs containing a particular item or itemset. This paper calls the designed pattern mining problem as the targeted mining of top- k high utility itemsets.

Unlike other HUIM problems, this paper needs to face the challenge of how to better deal with the search space under the combination of the target pattern and the top- k pattern. In other words, it needs to apply a reasonable and efficient data structure to store the itemset information and a correct and fast pruning strategy to improve the mining efficiency. So far, the relevant definitions and concepts have been described, and the next section will introduce the details of the proposed TMKU algorithm.

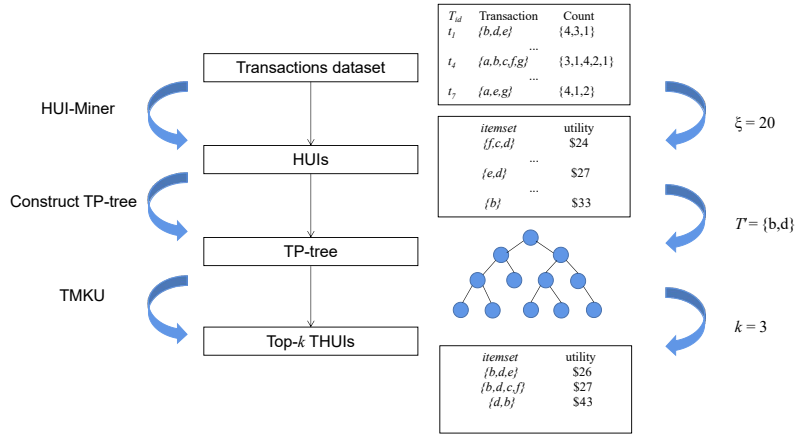


Figure 1: The framework of TMKU

4. The TMKU Algorithm

The targeted mining of top- k HUIs algorithm, abbreviated as TMKU, is presented at length in this section. Figure 1 is the framework diagram of the proposed TMKU algorithm, showing the basic steps of the whole algorithm. TMKU uses a trie structure as storage method similar to the TargetUM algorithm, and then utilizes the idea of filtering to dynamically change η (the utility threshold) to select the desired top- k THUIs. The TMKU algorithm enables a wide range of applications by adjusting the target pattern and k to query different targets and the number of result itemsets in a short period of time. After presenting these concepts, we will explain how search space pruning is conducted as well as the processes of the TMKU algorithm in detail.

4.1. Constructing the TP-tree and query mechanism

For data analysis, the database must be scanned to process the information and obtain patterns that meet the user's requirements. When using the target-pattern tree as a data structure, the algorithm uses nodes to represent itemsets, and the pattern tree can be updated incrementally by inserting new nodes.

MEIT [11] is an early study about this problem. It uses a compact node-compression mechanism to store information about transactions and answer queries about target itemsets, but it is not designed for utility mining. Later, the TargetUM algorithm was proposed for HUIM. The algorithm uses a trie tree to store HUIs from the whole database, and this was shown to reduce memory consumption. Therefore, the algorithm proposed in this paper also adopts the trie structure, which can reduce the overhead time for queries by using the common prefix of strings to improve efficiency. Then, after identifying all HUIs containing the target item or itemset, a map structure (TopKMap) is built to store the discovered target itemsets. The algorithm takes the utility value of the k -th itemset as the initial threshold, and constantly updates the TopKMap w.r.t. the minimum threshold until the top- k target HUIs are discovered.

Definition 7. (Target pattern tree). There are seven elements describing each node in this tree. For a node n (*name*), its parent node (*parent*) is recorded. In addition, the transaction weighted utilization (*twu*), *sumlu* which records the utility of the itemsets represented by the current node, and *sumRu* which records the remaining utility of the itemsets as the current node, are the three elements related to the utility. When n is the last item of a HUI, one variable (*isEnd*) is set to true. If not, *isEnd* is set to false. The last element indicates the link to the next node (*link*) containing the same item.

Definition 8. (Item header table). Each item has its own item header table. There is also a pointer to the first linked node corresponding to each item in the TP-tree. This table is useful for quickly locating the position of the item nodes.

When the initial HUI $\{f, c, d\}$ is found, the node f is first added into this tree since it has the smallest *TWU* value. Moreover, the element information of f is recorded. Then, the algorithm creates the item header table of the f node.

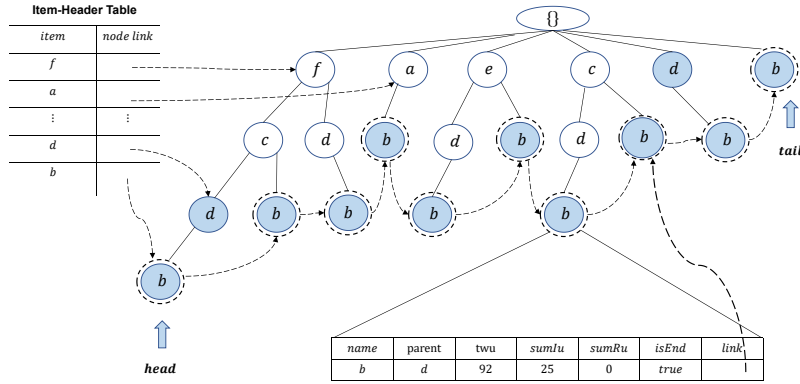


Figure 2: The construction of TP-tree

The other nodes are then inserted in the same way. However, each time a node is inserted, the algorithm needs to verify whether a node with the same name already exists. If it already exists, the node is not created again. But the inserted itemset will share a common prefix with the already existing node item. When the second HUI $\{f, c, d, b\}$ is inserted into the TP-tree, it shares the prefix (f, c, d) with the first HUI $\{f, c, d\}$. Then, node b is created, and its information is recorded to update the item header table. Furthermore, the update of the item header table for item b requires linking the new node. Subsequently, the last b becomes the updated trailing node. Other HUIs that are stored in the TP-tree must also be inserted using the same process until the construction of the TP-tree is finished. The construction of the TP-tree is illustrated in Figure 2.

4.2. Targeted utility raising strategy

As mentioned before, if only the value of k is set by the user, the task of top- k HUIM is to find out the top- k HUIs. Since the utility threshold is not given by the user, effective utility raising strategies need to be proposed to efficiently find the top- k HUIs. In this paper, the TMKU algorithm adopts two strategies (SUR and RIU) to achieve this goal.

Strategy 1. (SUR strategy). The SUR strategy [43] is a classic threshold raising strategy, used by the TMKU algorithm when storing the results in the TopKMap. The TopKMap can store k items and their utility values dynamically.

At first, the TopKMap is empty, and then when each THUI is found, the designed TMKU algorithm needs to consider whether to add it to the TopKMap or not. If the utility value of THUI is greater than η , it can be included in the TopKMap. However, when the number of itemsets stored in the TopKMap is greater than the k value set by the user, some itemsets in the TopKMap need to be gradually deleted and updated. Those itemsets with low utility values will be replaced by itemsets with high utility values.

Strategy 2. (RIU strategy). The RIU raising strategy [38] is primarily based on the real utilities of items. If the number of THUIs is more than k and the k -th highest utility value after sorting HUIs is still higher than η , the value of η should be changed to the k -th highest value.

The RIU threshold raising strategy plays an integral role in the TMKU algorithm. The true utility of each item can be identified by scanning THUIs. To raise the utility threshold more quickly, TMKU uses the true utility of the first item as the reference value for η . Finally, combining with the SUR strategy, the correct top- k THUIs can be quickly mined.

4.3. Pruning strategies

The TP-tree and item header table have been presented, which speeds up processing. Furthermore, some utility threshold-raising strategies are also adopted into the TMKU algorithm, which were listed in the previous subsection.

Some pruning strategies used in TMKU are introduced in this subsection. They allow the algorithm to considerably reduce the search space and time.

Strategy 3. If the sum of utilities of $X_i < \xi$, then X_i is not a THUI.

It was shown in prior work that if $TWU(X_i) < \xi$, $u(X_i)$ is also less than the threshold since $u(X_i)$ must be smaller than $TWU(X_i)$. Therefore, it is not necessary to check any itemsets containing X_i . The utility and remaining utility of each item are recorded in its utility-list. If the sum of the utilities of X_i ($sumIu$) is smaller than ξ defined by the user, then X_i is not the HUI that we are looking for. Conversely, it is a HUI. Moreover, the utility-list also records the sum of the remaining utilities called $sumRu$, which can be cleverly used to prune these utility values to improve search efficiency.

Strategy 4. In the TMKU algorithm, let X' and ξ be the candidate and current minimum utility value, respectively. Suppose that the sum of $sumIu$ and $sumRu$ of X' is smaller than ξ . Then the supersets and extended itemsets of X' do not need to be checked.

In other words, the sum of $sumIu$ and $sumRu$ respects the downward closure property. Therefore, no extension representing X' can be the required top- k THUI. This pruning strategy can prevent exploring several itemsets. It is worth noting that if the sum of $sumIu$ and $sumRu$ of X' is greater than ξ , then the extension of X' is a potential HUI and a further check is needed.

In addition, we can find that more and more branches may be generated during the construction of the TP-tree. However, the task is to discover all the top- k HUIs that contain a target itemset, so not all high-utility itemsets must be searched. The TMKU algorithm needs to determine if the target itemset is included before inserting a node to reduce the running time. Therefore, TMKU uses the item header table to cut off the useless branches in advance and correctly and quickly identify the high-utility itemsets containing the target pattern. Then, we can use the item header table to quickly find the nodes in the target pattern tree.

Strategy 5. When T' is set as the target pattern, the TMKU algorithm sorts these items of the target itemset in TWU ascending order. At this point, the item i with the largest TWU value is selected, and the position of i in the TP-tree and its branches is determined using the item header table. A bottom-up approach is used to query whether this HUI contains the target pattern. When determining whether item j in the target pattern is in the branch, we can compare the TWU of the current node with the value of $TWU(j)$. If the current node's TWU is smaller than $TWU(j)$, the branch does not contain the target pattern and can be discarded directly. If the current node's TWU is greater than $TWU(j)$, further exploration is required. If equal, it is also necessary to check whether the current node's node name is j to avoid replacing the same TWU value items.

For example, supposing that $\{f, c, d, b\}$ is a HUI, and $\{a, b\}$ is denoted as T' . First, we find that item b whose TWU is the highest in the target pattern. Then, we need to continue checking whether a is in the HUI, and then compare the TWU values of other items in the HUI with the TWU of a in turn. It is found that the TWU of both d and c is greater than a , but the TWU of f is less than a . According to the above analysis, it is known that $\{a, b\}$ is not in the HUI and the search is stopped.

4.4. Proposed algorithm

The TMKU task will be summarized as follows, based on the above introduction. To discover those itemsets containing the target itemset, Algorithm 1 focuses on accomplishing this task by constructing the TP-tree and the item header table. Algorithm 2 introduces how to search the TP-tree to find the THUIs followed by mining the top- k itemsets, and through some utility threshold raising strategies, users can get the desired results as soon as possible.

(1) The construction of TMKU: this procedure can be divided into two parts: the construction of the TP-tree and the processing of the item header table. The construction of a TP-tree is the basis of the TMKU algorithm. There are several indispensable parameters need to be introduced, including its last item x , the list of three utilities (IUs , Rus , TUs), and the three utility values of x . The *mapItemNode* is available to store the header node of each item header table. During the construction of the TP-tree, the node used to store the last item in each itemset is called the tail node, and the tail node is continuously updated. Besides, the *mapItemLastNode* is required to store the tail nodes of each item header table. First, the algorithm initializes the TP-tree, i.e., sets the initial values for the several parameters (line

Algorithm 1: The construction procedure

Input: X' : the prefix of HUI; IUs : the utility-list of X' ; RUs : the remaining utility-list of X' ; TUs : the TWU list of X' ; x : the last item of HUI; α : the utility of x ; β : the remaining utility of x ; θ : the TWU of x ; $mapItemNode$ store the header node of each item-header table; $mapItemLastNode$: store the tail nodes of each item-header table; ξ : the target utility threshold.

Output: TP -tree; item header table.

```

1 initialize  $TP$ -tree,  $currentNode = null$ ,  $parentNode = null$ ,  $listNodes = root.childs$ ;
2 insert a HUI into the  $TP$ -tree;
3 for each item  $e \in X'$  do
4     search the position of  $e$ ;
5     if  $currentNode == null$  then
6         create a new node in the  $TP$ -tree and construct item header table of  $e$ ;
7     end
8     update  $parentNode$ ,  $listNodes$ ;
9 end
10  $currentNode = x$ ;
11 if  $currentNode \neq null$  then
12     update  $currentNode.sumLu$ ,  $currentNode.sumRu$ ,  $currentNode.TWU$ ;
13 else
14     construct new item header table of  $x$ ;
15 end
16  $currentNode.isEnd = true$ ;
```

1). Based on the HUIs obtained by applying HUI-Miner, once a new HUI is found, we should consider inserting it into the TP -tree (line 2). For each item n in X' (line 3), the position of n in this TP -tree can be found easily (line 4). When there are no n nodes in the TP -tree, the TMKU algorithm creates a new node and stores its information in the item header table (lines 5–6). After that, the values of nodes $parentNode$ and $listNodes$ are updated (line 8). When all the items of X' have been processed, the TP -tree of X' is completely built (lines 3–8). Then, while x previously existed (found by the binary search method) (line 10), the algorithm will update the information value of the current node (lines 11 to 12). And if it does not exist, an item header table of x is created (line 14). Finally, the value of the $currentNode.isEnd$ variable is set to true (line 16).

(2) The discovery of TMKU: The procedure for processing the target query is a depth-first search. First, the procedure determines whether the received prefix appears. Therefore, the node is denoted the last item of T' , and then the $posToMatch$ parameter is initialized to record the current two items (line 1). Then, each item of T' is traversed. If the sum of $sumLu$ and $sumRu$ is not less than ξ , according to Strategy 4, X may be a THUI, and further comparison with other items can be performed (lines 2–25). Moreover, according to Strategy 5, if the $TWU(currentNode)$ is smaller than the current item of T' , it can be discarded directly, and the comparison is aborted (lines 9–10). When the value of $posToMatch$ decreases to -1, one of the target itemsets is found (lines 19–20). At this point, the algorithm uses a recursive method to output this THUI. First, all suffix nodes of X are explored, and from each node, the extension nodes are obtained (line 22). Then, the THUIs are stored by filtering out the high-utility itemsets that do not contain the target itemset. Finally, if the remaining suffix nodes meet the condition of Strategy 4, they need to be explored.

After that, the algorithm obtains the target high-utility itemsets. Then, the task of the algorithm is to quickly mine the top- k THUI. First, TopKMap is initialized to the empty set and the utility value of the first THUI is used to initialize η (line 26). Then, the algorithm iterates through all the itemsets in the THUIs and if the utility value of the itemset is higher than η , the itemset is added to the TopKMap (lines 27–29). Otherwise, the THUI is removed (line 31). When the size of the TopKMap is over k (line 33), η is updated to the k -th highest utility in the TopKMap (line 34). Moreover, the algorithm updates the TopKMap and keeps the k target itemsets with the highest utilities in the TopKMap (line 35). Finally, all the top- k THUIs can be explored and output successfully (line 38).

5. Performance Evaluation

The TMKU algorithm was evaluated by a series of tests on various types of datasets. By comparing the results, the correctness and efficiency of the TMKU algorithm are proved. The TMKU algorithm, as previously noted, is the first

Algorithm 2: The discovery procedure

Input: k : the number of THUIs; T' : the itemset obtain target; ξ : the target utility threshold; η : the minimum utility threshold.

Output: top- k THUIs.

```

1   $posToMatch = |T'|$ ,  $node$  = the target item of last item;
2  while  $node \neq null$  do
3      if  $node.sumLu + node.sumRu \geq \xi$  then
4           $node.name$  = the current HUI  $X$ ;
5           $posToMatch$  decrease 1, update  $currentNode = node.parent$ ;
6          while  $currentNode \neq null$  do
7              if  $posToMatch \geq 0$  then
8                  a new item  $y = T'[posToMatch]$ ;
9                  if  $currentNode.TWU < y.TWU$  then
10                     break;
11                 end
12                 if  $currentNode.TWU == y.TWU$  AND  $currentNode == y$  then
13                      $posToMatch$  decrease 1;
14                 end
15             end
16             update  $x$  and  $currentNode$ ;
17         end
18         if  $posToMatch == -1$  then
19             if  $node.sumRu \geq \xi$  AND  $currentNode.isEnd = true$  then
20                 a THUI has been discovered;
21             end
22             recursively explore all suffix nodes of  $X$ ;
23         end
24     end
25 end
26 initialize TopKMap =  $\emptyset$ ;
27 for each entry itemset  $t$  in THUIs do
28     if  $t.utility \geq \eta$  then
29         TopKMap.add( $t$ );
30     else
31         skip  $t$  from TargetUM;
32     end
33     if the number of TopKMap  $> k$  then
34         update  $\eta$  as the  $k$ -th highest utility in TopKMap;
35         update TopKMap and keep  $k$  target itemsets with the highest utilities in TopKMap;
36     end
37 end
38 return top- $k$  THUIs

```

to discover the top- k HUIs that include a target pattern. All the current top- k HUIM algorithms cannot successfully tackle the challenge of considering the user's needs. As a result, none of the existing methods can be compared to analyze and evaluate the proposed TMKU algorithm.

On the one hand, to determine the accuracy of the algorithm, we compared the results of the TMKU algorithm with those of the TargetUM algorithm after post-processing on various datasets. On the other hand, we conducted various experiments on six datasets to determine the efficiency of TMKU in a comprehensive manner and evaluated three variants using different pruning strategies. It is worth noting that $TMKU_{V1}$ only uses the Strategy 3 and Strategy 5, and $TMKU_{V2}$ uses the Strategy 4 and Strategy 5. When all pruning strategies are used, TMKU is obtained. As a result, three versions are used to evaluate the efficiency of TMKU.

Table 3

The characteristics of datasets

Dataset	Size	Trans	MaxLen	AvgLen
foodmart	176	4141	4.4	14
retail	1845	24735	10.3	74
ecommerce	1968	14976	11.6	29
chainstore	81102	1112950	7.2	170
mushroom	1309	8124	23	23
T10I4D100K	8267	100000	29	10.1

5.1. Experimental setup

All the algorithms were programmed in Java. The PC used for experiments is equipped with 8.0 GB of RAM, 64-bit Windows 10, and an AMD Ryzen 5 3600 CPU.

To assess the effectiveness and scalability of the TMKU algorithm, experiments were carried out on a variety of datasets, including real and synthetic databases. The SPMF data mining library is the source of all the datasets¹. These datasets have different characteristics, which makes it beneficial to observe the performance of the algorithm in different situations. Table 3 lists all the features of the experimental datasets. A brief description is given below.

- **Retail** is a real-life dataset with many items in each transaction. Transactions come from a store in Belgium and the utility values are synthetic.
- **Foodmart** is a transaction dataset from a grocery store where items are sparse and transactions are short.
- **Chainstore** is a transaction dataset containing over a million transactions from a chain store in California.
- **Ecommerce** is a transactional dataset that contains all the transactions of an online retail store and has real utility values.
- **Mushroom** is a dense and real-world database with over 8,000 transactions, where the *MaxLen* and *AvgLen* are the same, and utility values are synthetic.
- **T10I4D100K** is synthetic data generated by the IBM Data Generators, which contains 8,124 transactions.

5.2. Experiments on itemset

The output of the TMKU algorithm is the top- k THUIs. To evaluate the accuracy of the results, an experiment was done to compare the output of TMKU with that of TargetUM after post-processing the results. Here, u_1 represents the lowest utility in TMKU's result set, and u_2 denotes the same for the post-processed results from the TargetUM algorithm. We tested various k values for different datasets due to the different numbers and features of each dataset.

The results mainly demonstrate the effect of changing the k -value by comparing the minimum utility in the dataset. Table 4 indicates that u_1 and u_2 have the same value, which becomes smaller and smaller as k is increased. Besides, it is found that the results are the same for both algorithms. For example, we randomly select watermelon as the user-selected target pattern for the retail database, so T' is set as {976} which represents the watermelon. Then, the experiment compares the minimum utility value for different k values and u_1 and u_2 .

It is found that when k is increased from 10 to 110, the minimum utility value decreases from 4,325 to 1,389. TMKU described in this paper can successfully discover the top- k THUIs, depending on the aforementioned analysis. Table 5 displays the number of candidates in the six databases for different k values. For example, we set $T' = \{110\}$ for the mushroom database, which indicates the flavor of a certain type of mushroom. When k is increased from 1,000 to 6,000, the number of candidate itemsets increases from 16,978 to 65,512. As the k -value increases, obviously, the number of candidate itemsets increases as well. This is in line with the expected results of the top- k algorithm.

¹<http://www.philippe-fournier-viger.com/spmf/>

Table 4The minimum utility of TMKU with different k values

Dataset	u	# minimum utility by varying k					
		k_1	k_2	k_3	k_4	k_5	k_6
chainstore {16967}	u_1	1643,851	697,152	488,009	407,079	348,233	310,306
	u_2	1643,851	697,152	488,009	407,079	348,233	310,306
ecommerce {150561222}	u_1	976,618	954,154	940,042	929,362	920,962	913,834
	u_2	976,618	954,154	940,042	929,362	920,962	913,834
retail {976}	u_1	1,381	1,379	1,377	1,377	1,376	1,375
	u_2	1,381	1,379	1,377	1,377	1,376	1,375
foodmart {1340}	u_1	6,266	4,319	5,541	6,530	7,420	7,978
	u_2	6,266	4,319	5,541	6,530	7,420	7,978
mushroom {110}	u_1	449,193	358,323	256,158	125,079	93,238	73,171
	u_2	449,193	358,323	256,158	125,079	93,238	73,171
T10I4D100K {71}	u_1	93,238	39,273	30,843	27,984	26,228	25,716
	u_2	93,238	39,273	30,843	27,984	26,228	25,716

Table 5The number of candidates of TMKU with different k values

Dataset	# the number of candidates by varying k					
	k_1	k_2	k_3	k_4	k_5	k_6
chainstore {16967}	75	193	233	254	261	267
ecommerce {150561222}	43,509	84,230	123,914	160,464	196,604	231,791
retail {976}	4,430	7,771	10,557	13,064	15,193	17,242
foodmart {1340}	2,653	4,319	5,541	6,530	7,420	7,978
mushroom {110}	461	676	997	1248	1345	1456
T10I4D100K {71}	7,761	13,678	18,639	23,178	27,431	31,569

5.3. Experiments on runtime

Experiments were also done to compare the running times of several versions of the algorithm as k is changed. Figure 3 shows the trend of the execution time of the algorithm for various k values on the six datasets. It is worth noting that the top- k algorithm has only one parameter, which could result in varying performances. In general, for the TMKU task, it is tough to determine an appropriate value of k to find the optimal threshold that provides the best performance. As a result, the k -value in the TMKU algorithm is totally determined by the user's needs in most scenarios. This study is of great significance in targeted mining to consider the subjective needs of users and provide them with a positive experience.

The performance of the three algorithms on distinct datasets was recorded in the experiment to have a fair evaluation, which is depicted in Figure 3. The results reveal that when all the strategies are applied, the running time of the TMKU algorithm is shorter than that of other versions. The entire runtime increases as k grows. Due to the limitations of the TP-tree, the runtime of the three versions does not reflect significant differences. However, we can observe that this difference can be expressed in Figure 3 (a) and (d). Both the foodmart and chainstore datasets are sparse, with the chainstore dataset being particularly sparse. In different k settings, therefore, the running time difference of three variants can be clearly observed. The distinction between the retail database and the e-commerce database is not immediately obvious.

In summary, the algorithm performs better on sparse datasets with pruning strategies. For example, in the chainstore dataset, when $k = 210$, the proposed algorithm with all pruning strategies clearly takes less time than $TMKU_{V1}$ and $TMKU_{V2}$. TMKU takes 1,010 seconds to discover all the top- k THUIs, while $TMKU_{V1}$ and $TMKU_{V2}$ need 1,019 seconds and 1,025 seconds to finish this task, respectively. This result demonstrates that all the strategies are effective for mining the top- k THUIs. Furthermore, Strategy 3 and Strategy 4 work on the prefix and suffix of itemset, separately, which are crucial in speeding up the query. Strategy 5 has a pruning effect for the case when the target pattern $|T'| > 1$, as mentioned in the TargetUM task.

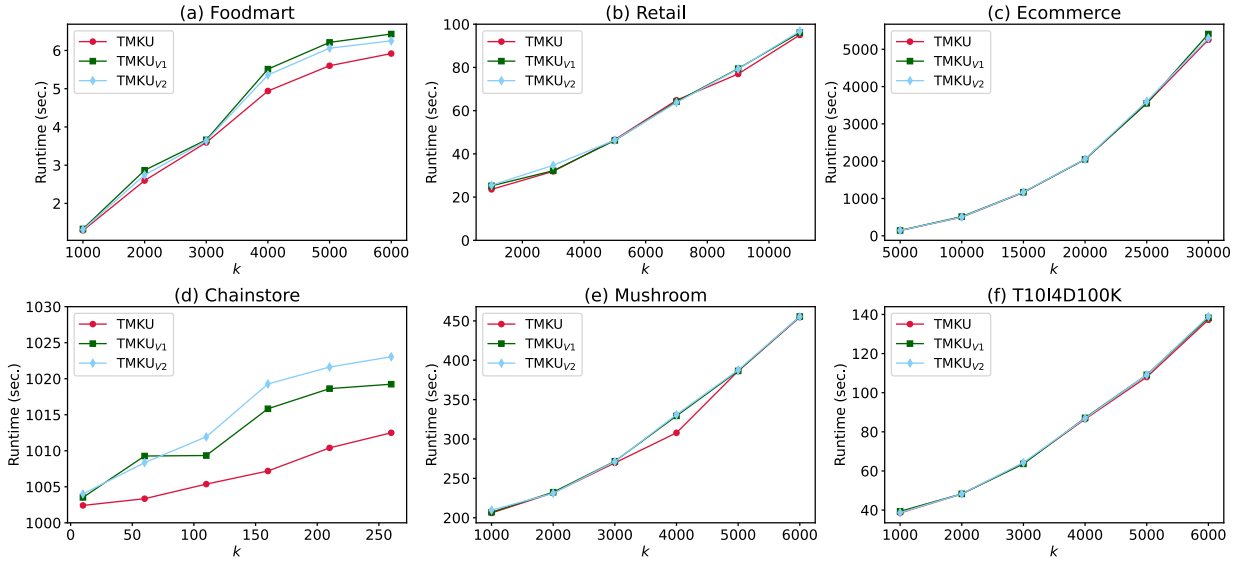


Figure 3: Runtime under varied k . (a) foodmart ($T' = \{1340\}$). (b) retail ($T' = \{1370\}$). (c) Ecommerce ($T' = \{150561222\}$). (d) Chainstore ($T' = \{16967\}$). (e) Mushroom ($T' = \{110\}$). (f) T10I4D100K ($T' = \{71\}$)

5.4. Experiments on memory

Additional tests were done to check how much memory is used by each version of the algorithm when the k parameter is varied, and Figure 4 shows the detailed results. We can clearly see that the algorithm that uses all pruning strategies consumes less memory than the other variants. This is due to the fact that some memory is consumed during the first phase of the TP-tree construction, but the memory impact of these pruning strategies is not significant. In addition, the overall memory usage of TMKU increases as k increases in all three versions, eventually reaching equilibrium. Since the top- k THUIs are mined by using the Strategy 1 and Strategy 2 to save the items and real utility values of items, more storage space is necessary in the second step of the algorithm. However, under certain conditions, the algorithm can fluctuate slightly in the amount of memory consumed while running. For example, for the foodmart dataset, as k is gradually increased from 1,000 to 6,000, the memory usage is also gradually increasing. However, when k is set to 4,000 in the T10I4D100K database, the memory usage is slightly less than before. In addition, the memory usage is more stable for datasets like chainstore where the number of transactions is high, and the utility values are concentrated. In general, the TMKU algorithm has pretty good capability in terms of memory usage.

5.5. Experiments on scalability

Finally, to better assess the performance of the TMKU algorithm, the scalability of TMKU was tested on the chainstore dataset. The size of the chainstore dataset was increased linearly from 100K to 1,100K. The experimental results in terms of execution time and memory for the three variants of the TMKU algorithm are shown in Figure 5 for the case of $k = 50$. Overall, the TMKU algorithm performs the best among the three algorithms. It can be clearly seen that as the number of transactions in the database increases, the execution time increases as well. For instance, when the chainstore database size grows from 300K to 900K, the execution time of TMKU increases from 242.86s to 923.79s. When compared to TMKU_{V1} and TMKU_{V2}, TMKU takes the least time. As far as memory usage is concerned, we can observe that memory consumption in general also increases as the data set size increases. However, the performance of strategies adopted by TMKU is subject to fluctuations due to the instability of memory consumption. When the dataset size is 500K, for example, TMKU's memory consumption differs very little from that of TMKU_{V1} and TMKU_{V2}. Figure 5 clearly shows that the runtime and memory consumption increase linearly with the size of the dataset. Besides, the TMKU algorithm is less time-consuming and space-consuming than the other two variants, so it has good scalability.

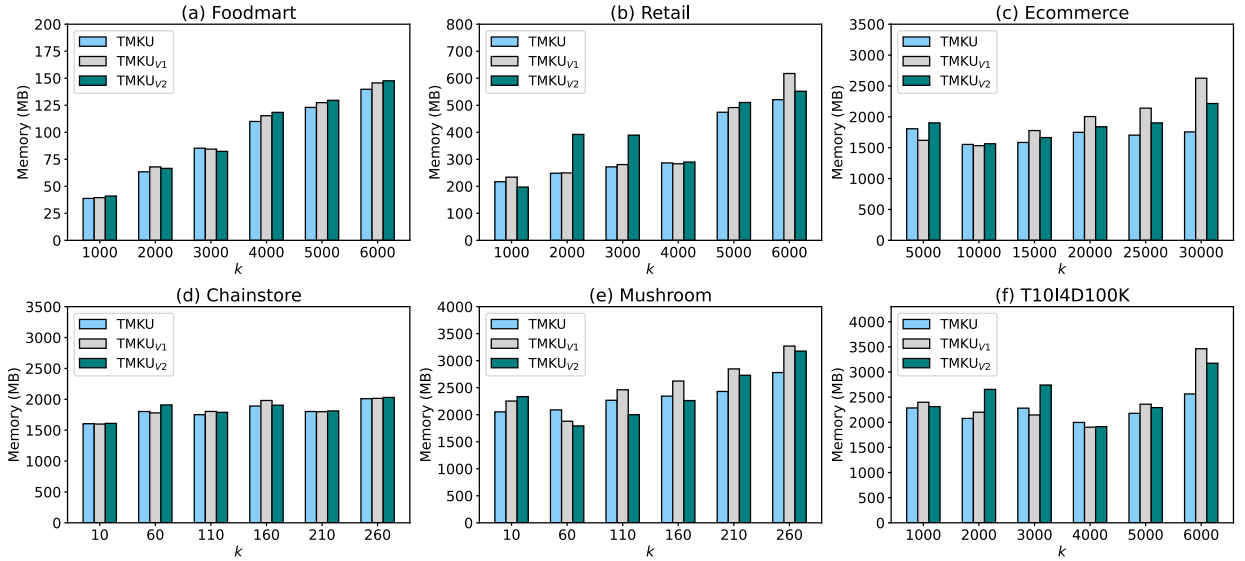


Figure 4: Memory for varied k values. (a) foodmart ($T' = \{1340\}$). (b) retail ($T' = \{1370\}$). (c) Ecommerce ($T' = \{150561222\}$). (d) Chainstore ($T' = \{16967\}$). (e) Mushroom ($T' = \{110\}$). (f) T10I4D100K ($T' = \{71\}$)

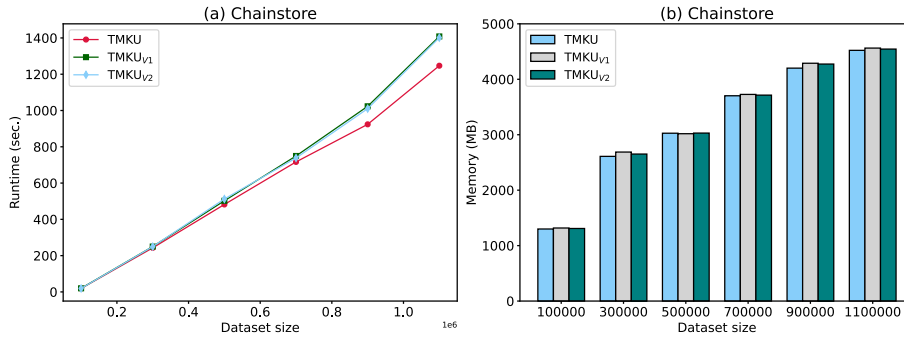


Figure 5: The runtime and memory scalability of variants on Chainstore ($T' = \{16967\}$, $k = 50$)

6. Conclusion

Until now, no algorithm has been proposed to mine the targeted top- k high utility itemsets. In this research, the TMKU algorithm is proposed for this task. The algorithm utilizes several data structures, namely the utility list, TP-tree, TopKMap, and two threshold-raising strategies (SUR and RIU) to achieve the task. In addition, several pruning strategies are applied in the TMKU algorithm to lower the execution time. Under this task, users can determine k and T' according to their needs. After testing the TMKU algorithm on six datasets, it was found that TMKU is able to create the TP-tree completely, raise the threshold quickly, and obtain the top- k THUIs efficiently. Moreover, TMKU has short processing time, stable memory usage, and excellent scalability. The TMKU algorithm presents a solution to a new problem in the field of data mining, and in the future, we will work on designing more advanced algorithms to explore this promising topic. Furthermore, to explore more applications of TMKU and extend it to more scenarios, such as using it in a distributed environment. Future research could also take into account visualization of mining results to boost interpretability.

Acknowledgment

This research was supported in part by the National Natural Science Foundation of China (Nos. 62002136, 62272196, 61472049, and 61572225), Natural Science Foundation of Guangdong Province (No. 2022A1515011861), Guangzhou Basic and Applied Basic Research Foundation (No. 202102020277), the Young Scholar Program of Pazhou Lab (No. PZL2021KF0023), Engineering Research Center of Trustworthy AI, Ministry of Education (Jinan University), and Guangdong Key Laboratory for Data Security and Privacy Preserving..

References

- [1] Aggarwal, C.C., Bhuiyan, M.A., Al Hasan, M., 2014. Frequent pattern mining algorithms: A survey, in: Frequent Pattern Mining. Springer, pp. 19–64.
- [2] Agrawal, R., Imieliński, T., Swami, A., 1993. Mining association rules between sets of items in large databases, in: Proceedings of the ACM SIGMOD International Conference on Management of Data, pp. 207–216.
- [3] Agrawal, R., Srikant, R., et al., 1994. Fast algorithms for mining association rules, in: The 20th International Conference on Very Large Data Bases, Citeseer. pp. 487–499.
- [4] Chai, W., 2009. Analyzes and solves the top enterprise network data security issues with the web data mining technology, in: First International Workshop on Database Technology and Applications, IEEE. pp. 312–314.
- [5] Chan, R., Yang, Q., Shen, Y.D., 2003. Mining high utility itemsets, in: Third IEEE International Conference on Data Mining, IEEE Computer Society. pp. 19–19.
- [6] Chand, C., Thakkar, A., Ganatra, A., 2012. Target oriented sequential pattern mining using recency and monetary constraints. International Journal of Computer Applications 45.
- [7] Cheng, H., Han, M., Zhang, N., Wang, L., Li, X., 2021. ETKDS: An efficient algorithm of top- k high utility itemsets mining over data streams under sliding window model. Journal of Intelligent & Fuzzy Systems , 1–22.
- [8] Chueh, H.E., 2010. Mining target-oriented sequential patterns with time-intervals. arXiv preprint arXiv:1009.0929 .
- [9] Duong, Q.H., Liao, B., Fournier-Viger, P., Dam, T.L., 2016. An efficient algorithm for mining the top- k high utility itemsets, using novel threshold raising and pruning strategies. Knowledge-Based Systems 104, 106–122.
- [10] Fournier-Viger, P., Gan, W., Wu, Y., Nouioua, M., Song, W., Truong, T., Duong, H., 2022. Pattern mining: Current challenges and opportunities, in: Proceedings of the 27th International Conference on Database Systems for Advanced Applications Workshops, Springer. pp. 34–49.
- [11] Fournier-Viger, P., Mwamkazi, E., Gueniche, T., Faghihi, U., 2013. MEIT: Memory efficient itemset tree for targeted association rule mining, in: International Conference on Advanced Data Mining and Applications, Springer. pp. 95–106.
- [12] Fournier-Viger, P., Wu, C.W., Zida, S., Tseng, V.S., 2014. FHM: Faster high-utility itemset mining using estimated utility co-occurrence pruning, in: International Symposium on Methodologies for Intelligent Systems, Springer. pp. 83–92.
- [13] Gan, W., Du, Z., Ding, W., Zhang, C., Chao, H.C., 2021a. Explainable fuzzy utility mining on sequences. IEEE Transactions on Fuzzy Systems 29, 3620–3634.
- [14] Gan, W., Lin, J.C.W., Chao, H.C., Fujita, H., Yu, P.S., 2019a. Correlated utility-based pattern mining. Information Sciences 504, 470–486.
- [15] Gan, W., Lin, J.C.W., Chao, H.C., Yu, P.S., 2019b. Utility-driven mining of high utility episodes, in: IEEE International Conference on Big Data, IEEE. pp. 2644–2653.
- [16] Gan, W., Lin, J.C.W., Fournier-Viger, P., Chao, H.C., Hong, T.P., Fujita, H., 2018. A survey of incremental high-utility itemset mining. Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery 8, e1242.
- [17] Gan, W., Lin, J.C.W., Fournier-Viger, P., Chao, H.C., Tseng, V.S., Yu, P.S., 2021b. A survey of utility-oriented pattern mining. IEEE Transactions on Knowledge and Data Engineering 33, 1306–1327.
- [18] Gan, W., Lin, J.C.W., Fournier-Viger, P., Chao, H.C., Yu, P.S., 2019c. A survey of parallel sequential pattern mining. ACM Transactions on Knowledge Discovery from Data 13, 1–34.
- [19] Gan, W., Lin, J.C.W., Fournier-Viger, P., Chao, H.C., Yu, P.S., 2020. HUOPM: High-utility occupancy pattern mining. IEEE Transactions on Cybernetics 50, 1195–1208.
- [20] Han, J., Pei, J., Yin, Y., 2000. Mining frequent patterns without candidate generation. ACM SIGMOD Record 29, 1–12.
- [21] Hemalatha, M., 2012. Market basket analysis—a data mining application in indian retailing. International Journal of Business Information Systems 10, 109–129.
- [22] Krishnamoorthy, S., 2015. Pruning strategies for mining high utility itemsets. Expert Systems with Applications 42, 2371–2381.
- [23] Krishnamoorthy, S., 2019. Mining top- k high utility itemsets with effective threshold raising strategies. Expert Systems with Applications 117, 148–165.
- [24] Kubat, M., Hafez, A., Raghavan, V.V., Lekkala, J.R., Chen, W.K., 2003. Itemset trees for targeted association querying. IEEE Transactions on Knowledge and Data Engineering 15, 1522–1534.
- [25] Kuriakose, S., Nedunchezian, R., 2017. Efficient adaptive frequent pattern mining techniques for market analysis in sequential and parallel systems. International Arab Journal of Information Technology 14, 175–185.
- [26] Li, Y.C., Yeh, J.S., Chang, C.C., 2008. Isolated items discarding strategy for discovering high utility itemsets. Data & Knowledge Engineering 64, 198–217.
- [27] Lin, J.C.W., Gan, W., Fournier-Viger, P., Hong, T.P., Tseng, V.S., 2016. Efficient algorithms for mining high-utility itemsets in uncertain databases. Knowledge-Based Systems 96, 171–187.

- [28] Liu, M., Qu, J., 2012. Mining high utility itemsets without candidate generation, in: Proceedings of the 21st ACM International Conference on Information and Knowledge Management, pp. 55–64.
- [29] Liu, Y., Liao, W.k., Choudhary, A., 2005. A two-phase algorithm for fast discovery of high utility itemsets, in: Pacific-Asia Conference on Knowledge Discovery and Data Mining, Springer. pp. 689–695.
- [30] Miao, J., Wan, S., Gan, W., Sun, J., Chen, J., 2021. Targeted high-utility itemset querying, in: IEEE International Conference on Big Data, IEEE. pp. 5534–5543.
- [31] Peek, N., Holmes, J.H., Sun, J., 2014. Technical challenges for big data in biomedicine and health: data sources, infrastructure, and analytics. Yearbook of Medical Informatics 23, 42–47.
- [32] Ryang, H., Yun, U., 2015. Top- k high utility pattern mining with effective threshold raising strategies. Knowledge-Based Systems 76, 109–126.
- [33] Shabtay, L., Yaari, R., Dattner, I., 2018. A guided FP-growth algorithm for multitude-targeted mining of big data. arXiv preprint arXiv:1803.06632 .
- [34] Song, W., Liu, Y., Li, J., 2014. Mining high utility itemsets by dynamically pruning the tree structure. Applied Intelligence 40, 29–43.
- [35] Song, W., Zhang, Z., Li, J., 2016. A high utility itemset mining algorithm based on subsume index. Knowledge and Information Systems 49, 315–340.
- [36] Sun, R., Han, M., Zhang, C., Shen, M., Du, S., 2021. Mining of top- k high utility itemsets with negative utility. Journal of Intelligent & Fuzzy Systems , 1–16.
- [37] Tseng, V.S., Shie, B.E., Wu, C.W., Yu, P.S., 2012. Efficient algorithms for mining high utility itemsets from transactional databases. IEEE Transactions on Knowledge and Data Engineering 25, 1772–1786.
- [38] Tseng, V.S., Wu, C.W., Fournier-Viger, P., Yu, P.S., 2015. Efficient algorithms for mining top- k high utility itemsets. IEEE Transactions on Knowledge and Data Engineering 28, 54–67.
- [39] Tseng, V.S., Wu, C.W., Shie, B.E., Yu, P.S., 2010. UP-Growth: an efficient algorithm for high utility itemset mining, in: Proceedings of the 16th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 253–262.
- [40] Wu, C.W., Shie, B.E., Tseng, V.S., Yu, P.S., 2012. Mining top- k high utility itemsets, in: Proceedings of the 18th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 78–86.
- [41] Wu, Y., Lei, R., Li, Y., Guo, L., Wu, X., 2021. HAOP-Miner: Self-adaptive high-average utility one-off sequential pattern mining. Expert Systems with Applications 184, 115449.
- [42] Zhang, C., Du, Z., Dai, Q., Gan, W., Weng, J., Yu, P.S., 2021a. TUSQ: Targeted high-utility sequence querying. arXiv preprint arXiv:2103.16615 .
- [43] Zhang, C., Du, Z., Gan, W., Yu, P.S., 2021b. TKUS: Mining top- k high utility sequential patterns. Information Sciences 570, 342–359.
- [44] Zhang, C., Du, Z., Yang, Y., Gan, W., Yu, P.S., 2021c. On-shelf utility mining of sequence data. ACM Transactions on Knowledge Discovery from Data 16, 1–31.
- [45] Zida, S., Fournier-Viger, P., Lin, J.C.W., Wu, C.W., Tseng, V.S., 2015. EFIM: a highly efficient algorithm for high-utility itemset mining, in: Mexican International Conference on Artificial Intelligence, Springer. pp. 530–546.